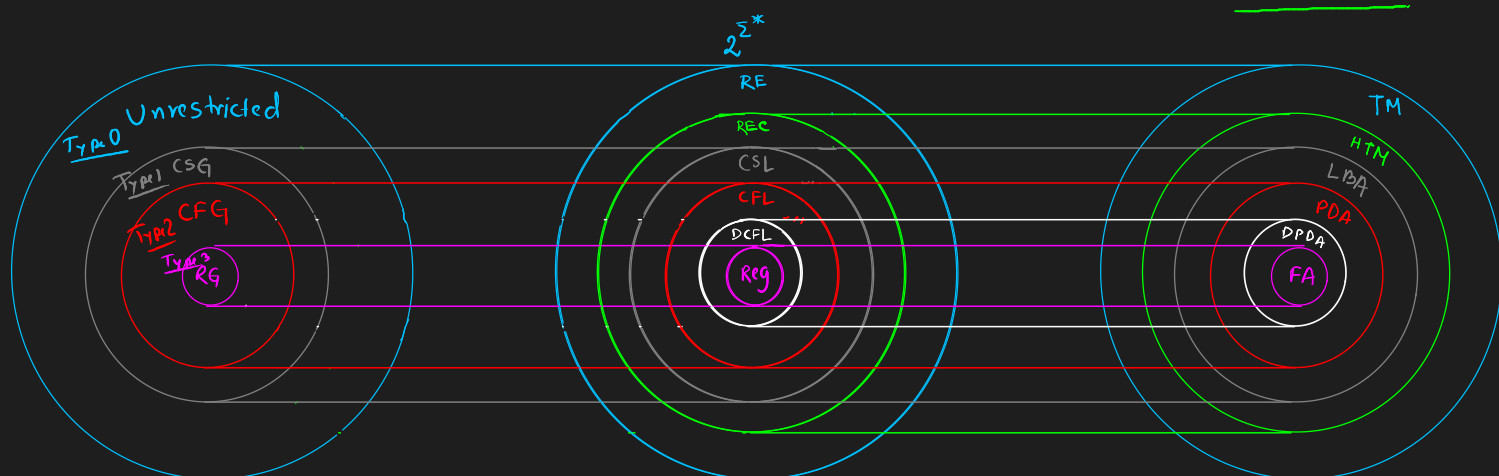


Language, Automata, Grammar

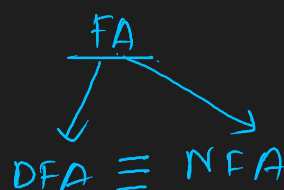
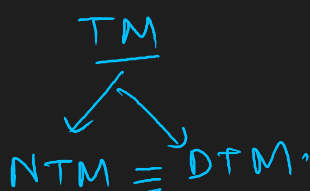
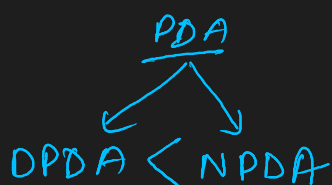
Grammar

Language

Automata



$$FA < DPDA < PDA < LBA < TM$$

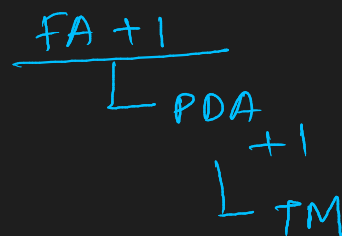


Which of the following is/are false?

- (I) $\{a^n b^n\} \cup \{a^n b^{2n}\}$ is CFL but not DCFL → True
 (ii) $a^m b^m c^n d^n$ is CFL but Not DCFL
 (iii) $\{a^m b^n \mid m, n \geq 0\}$ is DCFL not Regular

Options are

- a) I only
 b) I and II
 c) II and III
 d) I, II and III



Let $L(R)$ be language represented by Regular Expression R . Let $L(G)$ be language generated by Context free Grammar G . Let $L(D)$ be a language generated by deterministic Context free grammar. Which of the following is false?

- a) given a regular Expression R and a string w , is $w \in L(R)$ is decidable.
- b) given a Context free grammar G is $L(G)$ = finite or Infinite is decidable.
- c) Union of two $L(D)$ language is closed.
- d) Complement of $L(G)$ language is not closed.

Q 1. Consider $r = (11 + 111)^*$ over $\Sigma = \{0, 1\}$. Number of states in minimal NFA and DFA respectively:

(a) $N = 3, D = 4$ (b) $N = 3, D = 3$

(c) $N = 3, D = 3$ (d) $N = 4, D = 4$.

00:17

Q 2. Context free grammar is not closed under

(a) Product

(b) Union

(c) Complementation

(d) Kleen star

00:04

Q 3. A given grammar is called ambiguous if:

(a) Two or more productions have the same non-terminal on the left hand side

(b) A derivation tree has more than one associated sentence

(c) There is a sentence with more than one derivation tree corresponding to it

(d) Brackets are not present in the grammar

00:16

Q 4. Which of the following statements is (are) correct?

(a) Recursive languages are closed under complementation.

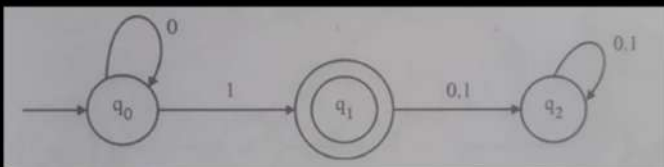
(b) If a language and its complement are both regular, the language is recursive

(c) Set of recursively enumerable language is closed under union

(d) All of these

00:12

Q 5. The finite state machine given in figure below recognizes:



(a) $L = \{0^n 1 : n \geq 0\}$

(b) $L = \{0^n 1 : n > 0\}$

(c) $L = \{0^n 1^n : n \geq 0\}$

(d) None of the

00:37

Q 6. Turing machine is more powerful than:

(a) Finite Automata

(b) Push Down Automata

(c) Both (a) and (b)

(d) None of these

00:32

Q 7. The context free grammar

$S \rightarrow SS \mid 0S1 \mid 1S0 \mid \epsilon$ generates:

- ☒ (a) Equal number of 0's and 1's
- (b) Unequal number of 0's and 1's
- (c) Any no. of 0's followed by any no. of 1's
- (d) None of these

00:36

Q 8. The language generated by the following grammar is:

$A_0 \rightarrow aA_1$

$A_1 \rightarrow ab$

- (a) Type 0
- (b) Type 1
- (c) Type 2
- ☒ (d) Type 3

00:37

Q 9. Recognize the CFL for the given CFG.

$S \rightarrow aB \mid bA$,

$A \rightarrow a|aS|bAA$,

$B \rightarrow b|bS|aBB$

01:13

- A. strings contain equal number of a's and equal number of b's.
- ☒ B. strings contain odd number of a's and odd number of b's.
- C. strings contain odd number of a's and even number of b's.
- D. strings contain even number of a's and even number of b's.

Q 10. Which of the following regular set does not contain the substring 101?

- a) $(0+1)^* 0^* 1^*$
- b) $(0+1)^* 10^*(0+1)^*$
- c) $(0+1)^* 0(00)^* 1$
- ☒ d) $(11)^* 0^*(01)^*$

00:33

Homomorphism $h(L)$ Substitution fn.

$$h(L) = \{ h(w) \mid w \in L \}$$

where $h: \Sigma \rightarrow \Gamma^*$ is called homomorphism.

$$\Sigma = \{0, 1\} \quad \Gamma = \{a, b\}$$

$$h(0) = aa, h(1) = bb \rightarrow \text{if } L = \{00, 101\}$$

$$\text{then } h(L) = \{aaaa, bbaabb\}$$

Inverse Homomorphism. $h^{-1}(L)$

$$\text{Let } h(0) = a, h(1) = b, h(2) = ab$$

$$\Sigma = \{0, 1, 2\}, \Gamma = \{a, b\}$$

$$\text{Let } L = \{abab\}$$

$$\text{then } h^{-1}(L) = \left\{ \begin{array}{l} 0101, \\ 2222, \\ 2012, \\ 0122 \end{array} \right\}$$

$$\text{If } h(0) = aa, h(1) = bb$$

$$\Sigma = \{0, 1\}, \Gamma = \{a, b\}$$

$$L = \{aa, aabb, baab, ababa\}$$

$$\therefore h^{-1}(L) = \{0, 01, 1\}$$

Decidability & Undecidability Table

Membership Problem $w \in \Sigma^*$	REG.	DCFL	CFL	CSL	REC	RE
Infiniteness Problem $L = \text{Infinte or finite}$	✓	✓	✓	✗	✗	✗
Emptiness Problem $L = \phi$	✓	✓	✓	✗	✗	✗
Equality Problem $L_1 = L_2$	✓	✓	✗	✗	✗	✗
L is Ambiguous?	✓	✓	✗	✗	✗	✗
Completeness $L = \Sigma^*$	✓	✓	✗	✗	✗	✗
$L_1 \cap L_2 = \phi$	✓	✗	✗	✗	✗	✗
if $L_1 \subseteq L_2$ Subset Problem	✓	✗	✗	✗	✗	✗

CFG, CFL.

CFG, CFL Introduction and Syllabus

- 1) Standard CFL's $\rightarrow$ Grammar & Properties
- 2) Derivation, Derivation Tree, Ambiguity
 $\swarrow \searrow$
 LMD RMD
- 3) Parsing, Membership Algo
 $\swarrow \searrow$
 Brute force CYK
- 4) Machines (DPDA, NPDA)*
- 5) DCFL Vs. CFL
- 6) Algorithm $\rightarrow$ Removal of λ Production
 $\rightarrow$ " " Unit "
 $\rightarrow$ " " Useless "
 $\rightarrow$ " " Left Recursion
 $\rightarrow$ " " Left factoring
- 7) Normal form $\rightarrow$ Chomsky Grammar

CFG $\rightarrow$ Context Free Grammar

CFG(V, T, P, S)

V: Finite Set of Variables (nonterminals)

T: Finite Set of Terminals ($V \cap T = \phi$)

P: Production Rules (Substitution Rules)

S: Start variable

Type 2

$\alpha \rightarrow \beta$
 $\downarrow$
 only 1 variable.
 $(V \cup T)^*$

$S \rightarrow 0S1$
 $S \rightarrow \epsilon$

or $S \rightarrow 0S1 \mid \epsilon$
 production rules.

$\{S\}$

$\{S\}$

$\{0, 1, \epsilon\} \rightarrow \text{Terminals}$

$S \rightarrow 0S1 \mid \epsilon$

$\downarrow$
 $0S1$
 $\downarrow$
 $0S1$
 $\downarrow$
 ϵ



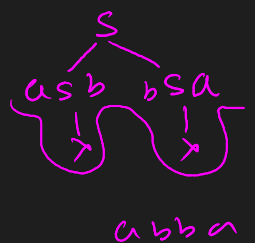
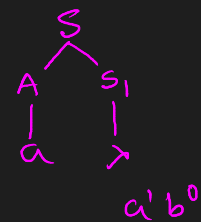
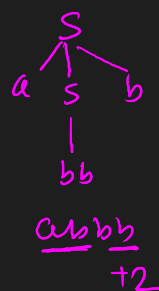
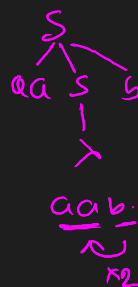
000111

$L = 0^n 1^n; n \geq 0$

Convert CFL $\rightarrow$ CFG

- 1) $a^n b^n, n \geq 0 \rightarrow S \rightarrow aSb \mid \lambda$
- 2) $a^n b^{n+2}, n \geq 0 \rightarrow S \rightarrow aSb \mid bb$
- 3) $a^{2n} b^n, n \geq 0 \rightarrow S \rightarrow aaSb \mid \lambda$
- 4) $a^{2n+3} b^n, n \geq 0 \rightarrow S \rightarrow aaSb \mid aaa$
- 5) $a^m b^n, m \geq n, m, n \geq 0 \rightarrow S_1 \rightarrow aS_1b \mid \lambda$
 $A \rightarrow aA \mid a, S \rightarrow AS_1$
- 6) $\{w \mid n_a(w) = n_b(w)\} \rightarrow S \rightarrow aSb \mid bSa \mid SS \mid \lambda$
- 7) $ww^R \cup w(a+b)w^R \rightarrow S \rightarrow aSa \mid bSb \mid a \mid b \mid \lambda$
- 8) $a^m b^m c^n, m, n \geq 0 \rightarrow S_1 \rightarrow aS_1b \mid \lambda$
 $c \rightarrow cC \mid \lambda, S \rightarrow S_1C$

if given $a^n b^n, n \geq 1 \rightarrow aSb \mid ab$



Derivation (Grammar $\rightarrow$ String)

LMD

RMD

$\downarrow$
 Left
 most
 Derivation

$\downarrow$
 Right
 most
 Derivation

$\mathcal{L}$

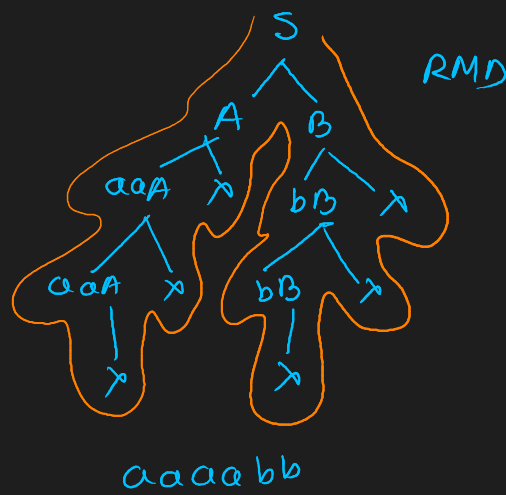
$S \rightarrow AB$

$A \rightarrow aaA \mid \lambda$

$B \rightarrow bB \mid \lambda$

Check whether

$w = aaaa bb \in L(G)$



PDA (Push Down Automata)

Formal Definition of PDA

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Q = finite set of states like FA

Σ = Input Symbol like FA

Γ = Stack Alphabet (Push into stack)

δ = Transition function, $\delta: Q \times (\Sigma \cup \epsilon) \times \Gamma \rightarrow Q \times \Gamma^*$

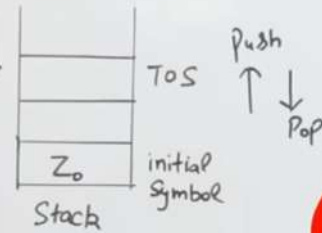
q_0 = initial state

Z_0 = Stack start symbol

F = final state



Finite Control



FA
+
Stack

PDA designing

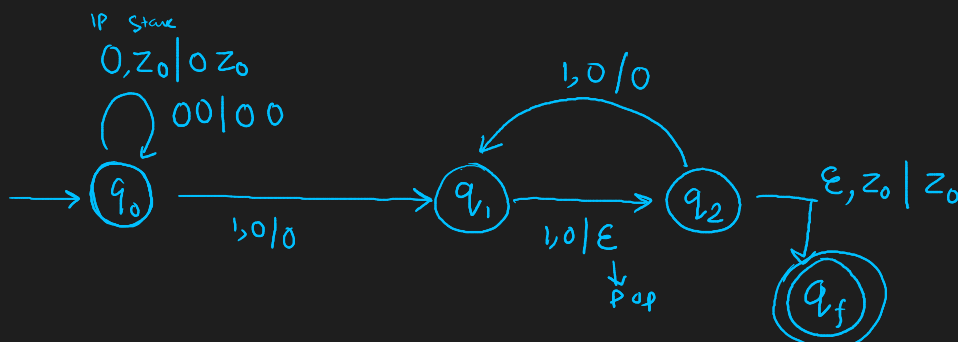
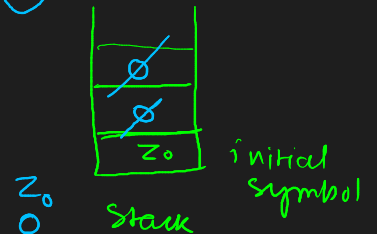
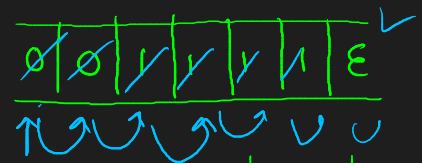
$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

$$L = \{0^n 1^{2n}, n > 0\}$$

$$= 011, 001111, 00011111, \dots$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{0, Z_0\}$$



$$q_0, 0, Z_0 \rightarrow 0Z_0$$

$$q_0, 0, 0 \rightarrow 00$$

$$q_0, 1, 0 \rightarrow q_1, 0$$

$$q_1, 1, 0 \rightarrow q_2, \epsilon$$

$$q_2, 1, 0 \rightarrow q_1, 0$$

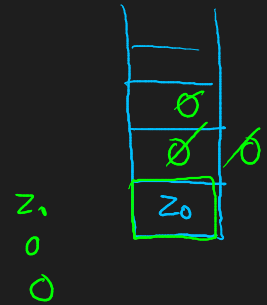
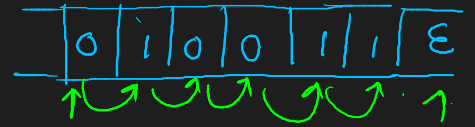
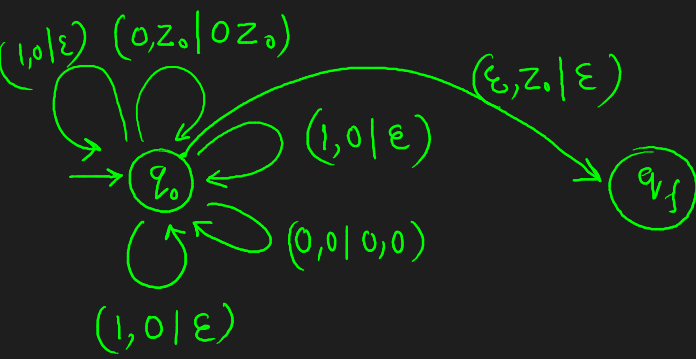
$$q_2, \epsilon, Z_0 \rightarrow q_f, Z_0$$

Design PDA

$$\delta = Q \times (\Sigma \cup \epsilon) \times \Gamma \rightarrow Q \times \Gamma^*$$

$$L = \{ w \in (0,1)^* \mid n_0(w) = n_1(w) \}$$

01, 10, 00, 11, 0011
 ✓ ✓ ✗ ✗ ✓



Closure Properties of CFL

Union ✓
 Concatenation ✓
 Kleen Closure ✓
 Intersection ✗
 Complementatation ✗

$$L_1 \rightarrow G_1$$

$$S_1 \rightarrow a s_1 b \mid \lambda$$

$$L_2 \rightarrow G_2$$

$$S_2 \rightarrow b s_2 c \mid \lambda$$

$$L_1^* \rightarrow G_1 \rightarrow S_1$$

$$S \rightarrow S_1 / S_2 \text{ (Union)}$$

$$S \rightarrow S_1 S_2 \mid \lambda$$

$$S \rightarrow S_1 S_2 \text{ (Concatenation)}$$

(Kleen Closure)

$$L_1 \cap L_2$$

$$a^n b^n c^m \cap a^m b^n c^n$$

$$\hookrightarrow a^n b^n c^n \rightarrow \text{CSL}$$

For all the CFLs closed.

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}} = \overline{\text{CFL} \cup \text{CFL}} = \overline{\text{CFL}} = \text{CFL}$$

Elimination of Null (ϵ) Production

$$S \rightarrow aS \mid A$$

$$A \rightarrow \epsilon$$

- ① Find Nullable variables
- ② Replace nullable variables in the right side with ϵ

Nullable variables = $\{ S, A \}$

$S \rightarrow aS \mid A \mid a \mid \epsilon \rightarrow$ now, A is removed, so remove that from the expression.

$$\therefore S \rightarrow aS \mid a \mid \epsilon$$

$S \rightarrow ABC$
 $A \rightarrow aA \mid \epsilon$
 $B \rightarrow bB \mid \epsilon$
 $C \rightarrow c$

Nullable $\rightarrow \{A, B\}$

$S \rightarrow ABC \mid Bc \mid AC \mid C$
 $A \rightarrow aA \mid \epsilon \mid a \rightarrow aA \mid a$
 $B \rightarrow bB \mid \epsilon \mid b \rightarrow bB \mid b$
 $C \rightarrow c$

$S \rightarrow ABC \mid Bc \mid AC \mid C$
 $A \rightarrow aA \mid a$
 $B \rightarrow bB \mid b$
 $C \rightarrow c$

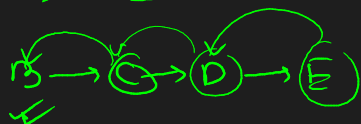
Elimination of Unit Production.

Q1

$S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow c \mid d$
 $C \rightarrow D$
 $D \rightarrow E$
 $E \rightarrow a$

Unit Production

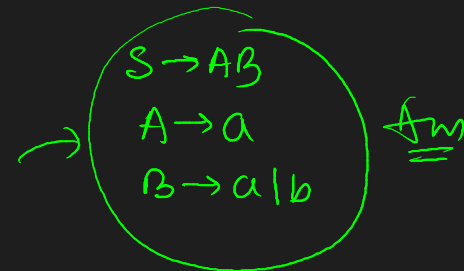
$B \rightarrow C$
 $C \rightarrow D$
 $D \rightarrow E$



$\alpha \rightarrow \beta$
 $\alpha, \beta \in V$

Unit Production.

$S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow a \mid b$
 $C \rightarrow a$
 $D \rightarrow a$
 $E \rightarrow a$



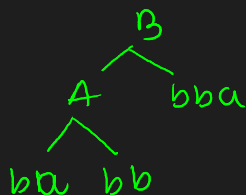
Unreachable.

Q2

$S \rightarrow aA \mid B$
 $A \rightarrow ba \mid bb$
 $B \rightarrow A \mid bba$

Unit Production

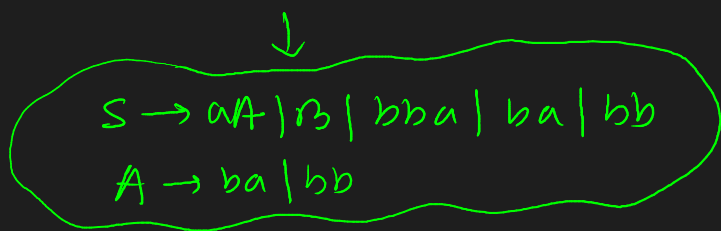
$S \rightarrow B \rightarrow A$



$S \rightarrow aA \mid B \mid bba \mid ba \mid bb$
 $A \rightarrow ba \mid bb$

$B \rightarrow bba \mid ba \mid bb$

Ans



Turing Machine

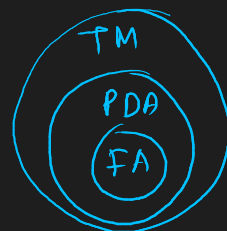
$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

$\Sigma \subseteq \Gamma$

$\delta = Q \times \Gamma \rightarrow Q \times \Gamma \times (L, R)$



• Read, Write
 • Left, Right



LBA (Linear Bounded Automata)

FA < PDA < LBA < TM

	\$	a	b	a	b	\$	
--	----	---	---	---	---	----	--

Standard Examples

- 1) $L = \{a^n b^n c^n : n \geq 1\}$
- 2) $L = \{a^n : n \text{ is prime}\}$
- 3) $L = \{a^n : n \text{ is non prime}\}$
- 4) $L = \{a^n! : n \geq 0\}$
- 5) $L = \{ww : w \in \{a,b\}^+\}$
- 6) $L = \{www^R : w \in \{a,b\}^+\}$
- 7) $L = \{w^n : w \in \{a,b\}^+, n \geq 1\}$
- 8) $L = \{a^n : n = m^2, m \geq 1\}$

Design Turing Machine for $\{a^n b^n | n \geq 1\}$

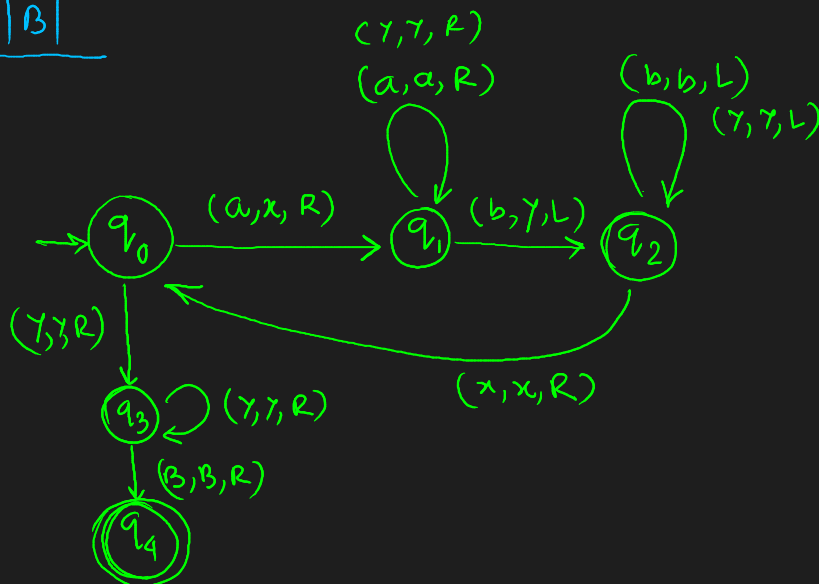
B		a		a		b		b		B
---	--	--------------	--	--------------	--	--------------	--	--------------	--	---

↑

B	B	a	a	a	b	b	b	B	B
---	---	--------------	--------------	--------------	--------------	--------------	--------------	---	---

↑ R/W
[FC]

~~a~~ ~~a~~ ~~b~~ ~~b~~ ~~b~~

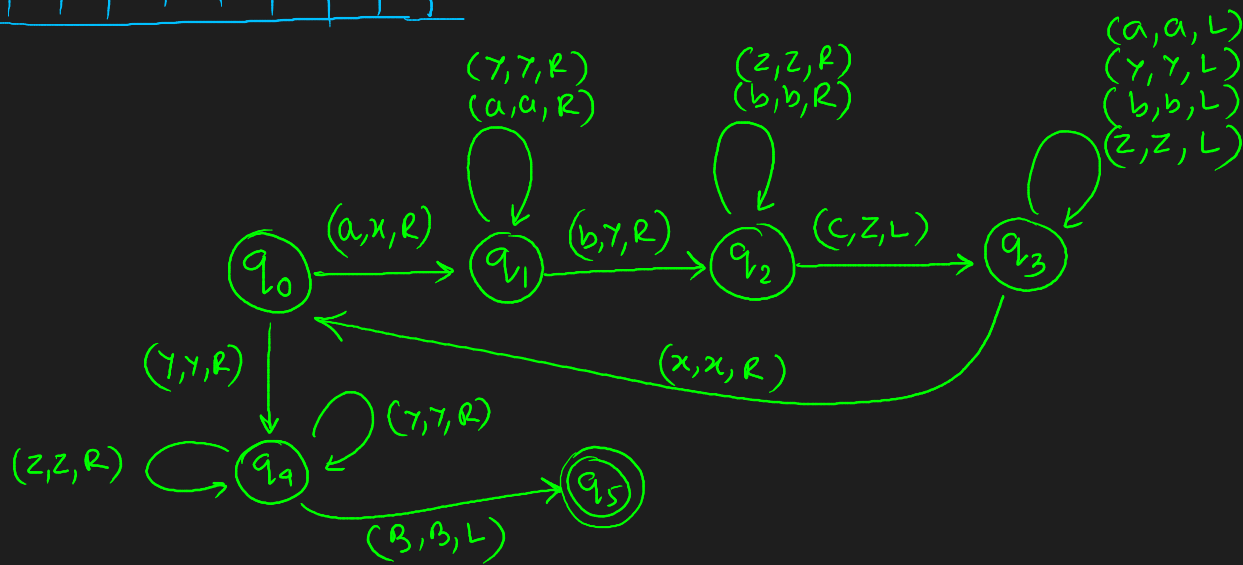


8. Design TM for $\{a^n b^n c^n | n \geq 1\}$

~~a~~ ~~a~~ ~~b~~ ~~b~~ ~~c~~ ~~c~~ ~~c~~

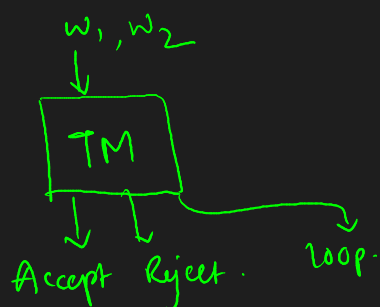
B	B	a	a	a	b	b	b	c	c	c	B	B
---	---	---	---	---	---	---	---	---	---	---	---	---

↑
[FC]



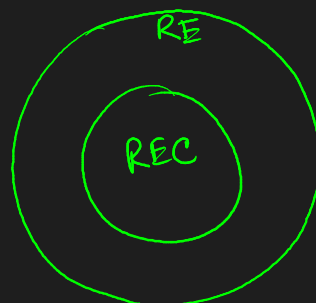
Recursive Enumerable Language

- L is RE if there is TM
- Three states: Halt and Accept
Halt and Reject
Never Halt ✓
- Closed under all except -, Complement

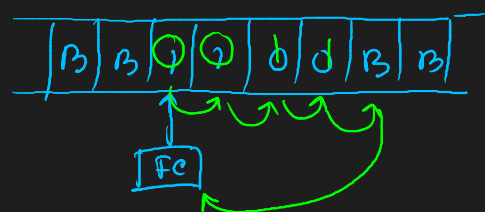


Recursive Language

- L is Recursive if there is Halting/Total TM
- Two states: Halt and Accept
Halt and Reject
- Closed under all except Homomorphism and Substitution

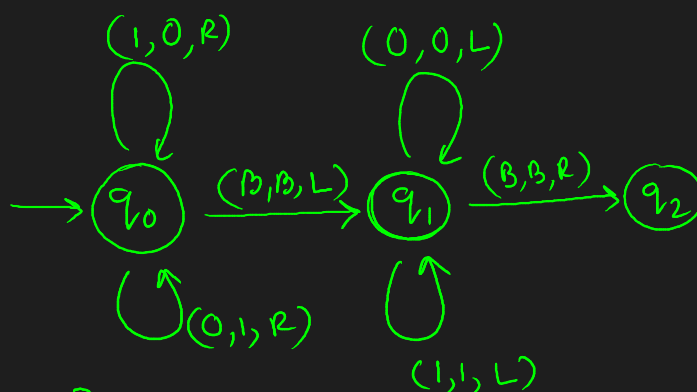


Turing Machine For 1's complement



States	0	1	1
q_0	1R q_0	0R q_0	1L q_1
q_1	0L q_1	1L q_1	1R q_2
q_2	-	-	-

Change direction
New State.



Turing Machine With Modifications

- 1) TM with Stay option
- 2) TM with Semi-infinite tape
- 3) Off-line TM
- 4) Multidimensional TM
- 5) NDTM
- 6) UTM
- 7) Multitape TM
- 8) Multi-head TM
- 9) TM with 3 states
- 10) Jumping TM
- 11) Non Erasing TM
- 12) Always Writing TM

- 1) TM without Writing Capacity (FA)
- 2) TM with Input Size Tape (LBA)
- 3) TM with Tape used as Stack* (NDTM) (PDA)
- 4) TM with finite tape. (FA)

FA < PDA < LBA < TM
 ↓
 DPDA < NPDA

CYK Algorithm: Check wheathere a string 'abbb' is a valid member of following CFG.

$S \rightarrow AB$
 $A \rightarrow BB/a$
 $B \rightarrow AB/b$

valid.
 abbb
 1 2 3 4

Starting symbol contains 'S'

	4	3	2	1
1	S, B	A	S, B	A
2	S, B	A	B	
3	A	B		
4	B			

14
 12 34
 SB A
 SA
 SB, BB
 11 24
 12 44 → =

12
 (11) (22)
 A B
 23
 (22) (33)
 B B
 34
 (33) (44)
 B B

24
 234
 22 34
 B A
 23 44
 A B

13
 abb
 123
 (11)(23)
 A φ - X
 (12) (33)
 (SB) B
 A - BB SB φ

* CYK applicable on CNF only

CNF: $A \rightarrow BC$ OR $A \rightarrow a$

* It is Universal (applicable on all CNF)

* Time Complexity $O(n^3)$

* Space Complexity $O(n^2)$

CNF

(Chomsky Normal Form)

1) $A \rightarrow BC$
 OR
 $A \rightarrow a$
 $\{A, B, C \in \text{Variables}\}$
 $\{a \in \text{Terminal}\}$

2) No. of steps required to generate a string of length 'n' is $2|n| - 1$

3) Used in Membership Algo.

4) Derivation or Parse tree obtained from CNF is always Binary Tree.

5) Length of each production Restricted

GNF

(Greibach Normal Form)

1) $A \rightarrow aX$
 where $\{X \in V^*\}$
 $\{a \in T\}$

2) No. of steps required to generate a string of length 'n' is 'n'

3) Used to Convert CFG to PDA

4) Not always

5) Not Restricted

Derivation Tree/Parse Tree in TOC & Compiler

• It is basically the sequence of steps followed to generate a Word(String) from a given Grammar.

G: $E \rightarrow E+E \mid E * E \mid E=E \mid id$

W: $id + id * id$

$x + y * z$

$id + id * id$

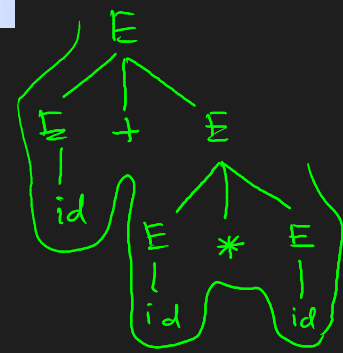
$E \rightarrow E + E$

$E + E * E$

$id + id * id$

$\therefore W(S) \in L(G)$

Derivation



- Recursive CFG generates Infinite number of strings.

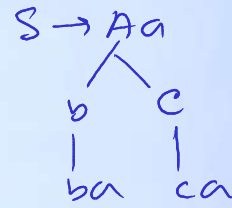
for example:

- 1) $S \rightarrow Sa$
 $S \rightarrow b$
 $\{b, ba, bba, bbb a, \dots\}$
- 2) $S \rightarrow aS$
 $S \rightarrow b$
 $\{b, ab, aab, aaab, \dots\}$
- 3) $S \rightarrow Aa$
 $A \rightarrow Ab | c$

- Non-Recursive CFG generates finite number of strings.

For example:

- $$S \rightarrow Aa$$
- $$A \rightarrow b | c$$



Ambiguous & Unambiguous Grammar

- For **ambiguous** there exists **more than one** derivation for any word that belongs to Grammar.
- For **unambiguous** there exist **exactly one** derivation for any word that belongs to Grammar.

For example: Given that $G: S \rightarrow Sa | aS | a$



→ Ambiguous.

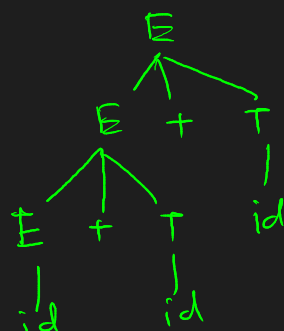
Conversion of Ambiguous to Unambiguous

$G: E \rightarrow E + E | id$

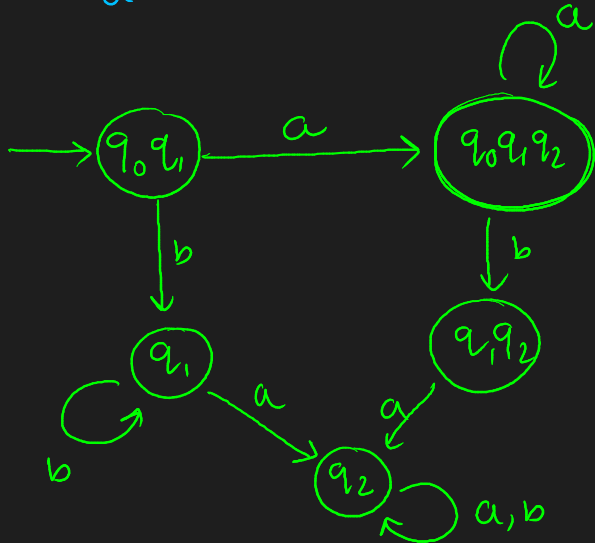
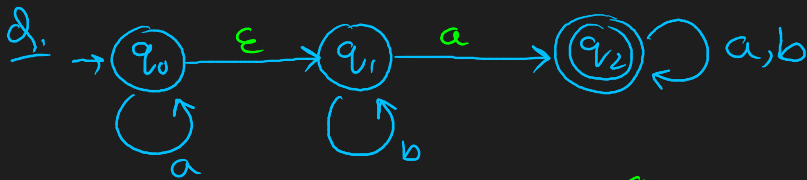
Where $w = id + id + id$

$E \rightarrow E + T | T$

$T \rightarrow id$



Convert ϵ -NFA $\rightarrow$ DFA



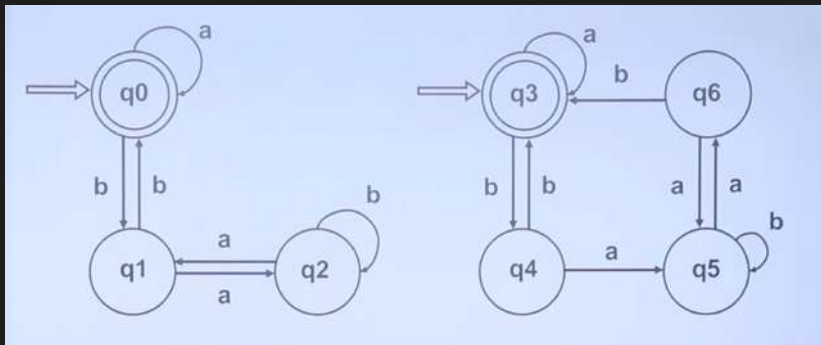
① ϵ -closure = q_0q_1 of q_0

② $q_0q_1 \xrightarrow{a} a(q_0) \cup a(q_1)$
 $q_0q_1q_2 \cup q_2$
 $= q_0q_1q_2$

$q_0q_1 \xrightarrow{b} q_1 \cup q_1 = q_1$

$q_0q_1q_2 \xrightarrow{a} q_0q_1q_2$
 $\xrightarrow{b} q_1q_2$

Equivalence of DFA



DFA1

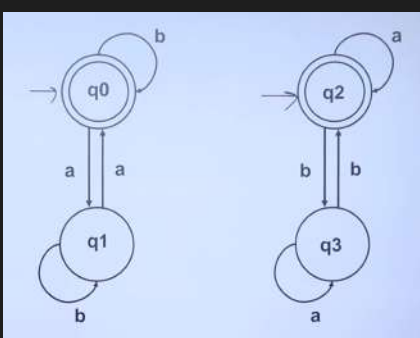
DFA2

$\therefore$ We can say that all states matched.
DFA1 $\equiv$ DFA2

Steps

- ① Same Initial & Final state $\checkmark$
- ② (q_0q_3) transition

	a	b
(q_0q_3)	q_0q_3 FS FS	q_1q_4 IS IS
(q_1q_4)	q_2q_5 IS IS	q_0q_3 FS FS
(q_2q_5)	q_1q_6 IS IS	q_2q_5 IS IS
(q_1q_6)	q_2q_5 IS IS	q_0q_3 FS FS



DFA1

DFA2

- ① Initial & final state same $\checkmark$
- ② (q_0q_2)

	a	b
(q_0q_2)	q_1q_2 IS FS X	q_0q_3 FS IS X

$\therefore$ DFA1 $\neq$ DFA2

Conflict